
Algoritmo de Metrópolis Passo a Passo

— *Simulação Numérica do Modelo de Ising* —

Iniciação Científica

Sumário

1	Introdução	1
2	A Física do Algoritmo	1
2.1	Onde a Temperatura entra?	1
2.2	A Regra de Ouro (E cadê a Função de Partição?)	2
3	Desenvolvimento do Código Fonte	3
3.1	Gerando o Retículo Primário em Desordem	4
3.2	Cálculo da Variação de Energia (ΔE)	5
3.3	Algoritmo de Metrópolis: Implementação	5
3.4	Escala Temporal: Passo de Monte Carlo (PMC)	6
4	Resumo	8

1 Introdução

Quando queremos simular o Modelo de Ising no computador (por exemplo, para calcular a magnetização ou a energia do sistema), esbarramos em um grande problema: como a rede é formada por vários spins que podem assumir valores para cima ou para baixo, o número de configurações possíveis cresce exponencialmente.

Para uma rede relativamente pequena de 16×16 spins ($N = 256$), existem 2^{256} configurações diferentes. Esse número é tão grande que torna inviável calcular exatamente as propriedades do sistema considerando todos os estados possíveis.

Para contornar essa dificuldade, utilizamos os **métodos de Monte Carlo**, sendo o mais conhecido o **algoritmo de Metrópolis**. Em vez de calcular médias sobre todas as 2^{256} configurações, o algoritmo realiza uma amostragem do espaço de estados.

A cada passo, ele seleciona um spin aleatoriamente e decide se deve invertê-lo com base em um critério probabilístico. Repetindo esse procedimento muitas vezes, o método gera configurações representativas do equilíbrio térmico, permitindo estimar corretamente as grandezas físicas de interesse.

2 A Física do Algoritmo

Antes de implementarmos o código em C/C++, é importante entender dois aspectos fundamentais de como o algoritmo incorpora os efeitos da temperatura e das probabilidades.

2.1 Onde a Temperatura entra?

No modelo teórico, assumimos que o sistema de spins está em contato com um reservatório térmico a temperatura constante. O algoritmo de Metrópolis simula exatamente essa situação de "banho térmico".

Por isso, na simulação, **a temperatura T é fixada no início e permanece constante durante toda a evolução do sistema.**

O fluxo típico da simulação segue a seguinte lógica:

1. Escolhemos um valor para a temperatura T .
2. Executamos o algoritmo de Metrópolis por várias iterações. Nessa fase, o sistema evolui sob a influência térmica definida e tende a atingir um estado de equilíbrio.

Esse processo é chamado de **termalização**.

3. Após atingir o equilíbrio, iniciamos a etapa de coleta de dados, acumulando grandezas físicas como a magnetização ao longo do tempo.

E como construímos, por exemplo, o gráfico da magnetização em função da temperatura? Repetimos todo o procedimento acima para diferentes valores de T . Ou seja, utilizamos um laço externo (um "super loop") no qual, para cada temperatura, o sistema é reinicializado, termalizado e, em seguida, suas propriedades são medidas.

2.2 A Regra de Ouro (E cadê a Função de Partição?)

Na mecânica estatística, a probabilidade de um sistema estar em uma configuração de energia E é dada pela distribuição de Boltzmann:

$$P(E) = \frac{e^{-\beta E}}{Z},$$

onde $\beta = \frac{1}{k_B T}$ e Z é a chamada **função de partição**.

A função de partição é definida como a soma sobre todas as configurações possíveis do sistema. No caso do modelo de Ising, isso envolve um número extremamente grande de estados (por exemplo, 2^{256} para uma rede 16×16), tornando seu cálculo direto computacionalmente inviável.

A ideia central do algoritmo de Metrópolis é contornar essa dificuldade trabalhando com **probabilidades relativas**, em vez de probabilidades absolutas.

Ao invés de avaliar toda a rede, consideramos apenas a mudança local associada à tentativa de inversão de um spin. Comparamos então a probabilidade do estado novo (n) com a do estado antigo (a):

$$\frac{P_{novo}}{P_{antigo}} = \frac{\frac{e^{-\beta E_n}}{Z}}{\frac{e^{-\beta E_a}}{Z}}.$$

Observa-se que a função de partição Z cancela exatamente nessa razão, resultando em:

$$\frac{P_{novo}}{P_{antigo}} = \frac{e^{-\beta E_n}}{e^{-\beta E_a}} = e^{-\beta(E_n - E_a)} = e^{-\beta \Delta E}.$$

Portanto, o algoritmo depende apenas da variação de energia ΔE associada à mudança local, sem necessidade de conhecer explicitamente Z .

É a partir dessa relação que derivamos as regras probabilísticas utilizadas no algoritmo de Metrópolis.

- **Quando $\Delta E \leq 0$:** Nesse caso, a inversão do spin leva a uma redução da energia do sistema ou, no pior cenário, não a altera. Como sistemas físicos tendem a estados de menor energia, a mudança é sempre aceita. Portanto, a inversão é realizada com probabilidade igual a 1 (aceitação determinística).
- **Quando $\Delta E > 0$:** Aqui, a inversão do spin aumentaria a energia do sistema, o que é energeticamente desfavorável. No entanto, devido às flutuações térmicas a temperatura finita, tais aumentos de energia podem ocorrer com certa probabilidade. Para modelar esse efeito, introduzimos um critério estocástico: sorteamos um número aleatório $\alpha \in [0, 1]$ com distribuição uniforme e comparamos com o fator de Boltzmann $e^{-\beta\Delta E}$.

A inversão do spin é aceita se

$$\alpha < e^{-\beta\Delta E},$$

e rejeitada caso contrário. Assim, mesmo transições energeticamente desfavoráveis podem ocorrer, mas com probabilidade exponencialmente pequena em função de ΔE .

O algoritmo de Metrópolis consiste essencialmente em um procedimento estocástico de amostragem no qual as transições de estado são condicionadas por critérios físicos bem definidos: a variação de energia do sistema e as flutuações térmicas, incorporadas pelo fator de Boltzmann. Dessa forma, o método combina regras determinísticas e probabilísticas para gerar configurações distribuídas de acordo com o equilíbrio térmico, sendo plenamente adequado para implementação em simulações numéricas.

3 Desenvolvimento do Código Fonte

Agora que entendemos como o problema pode ser tratado de forma eficiente por meio das variáveis do sistema, podemos avançar para a implementação computacional sem grandes dificuldades conceituais. Para simplificar as expressões, adotaremos unidades naturais, fixando $k_B = 1$.

A implementação será feita de forma modular, utilizando funções auxiliares responsáveis pelos sorteios aleatórios necessários ao algoritmo de Metrópolis. Para isso, utilizaremos funções da biblioteca padrão da linguagem (com destaque para `stdlib` e `math.h`):

```
1  #include <math.h>
2  #include <stdlib.h>
3
4  // Retorna um numero aleatorio em ponto flutuante no
   // intervalo [0,1]
5  // (nosso parametro probabilistico alpha)
6  float rand_float() {
7      return (float)rand() / (float)RAND_MAX;
8  }
9
10 // Retorna um indice inteiro aleatorio entre 0 e N-1,
11 // correspondente a um sitio da rede
12 int rand_sitio() {
13     return rand() % N;
14 }
```

3.1 Gerando o Retículo Primário em Desordem

Em muitos problemas de física estatística, é conveniente iniciar o sistema em um estado completamente desordenado. Esse cenário corresponde, na prática, ao limite de temperatura infinita ($T \rightarrow \infty$), no qual não há preferência energética entre as configurações possíveis.

Exercício 1 — Inicialização Aleatória da Rede

Implemente uma função `void inicializa_rede()` responsável por atribuir valores iniciais aos spins do sistema.

Percorra todos os sítios da rede, indexados por $i = 0, 1, \dots, N-1$. Para cada sítio, sorteie um número aleatório α no intervalo $[0, 1]$ utilizando a função `rand_float()`.

Adote o seguinte critério:

- Se $\alpha < 0.5$, atribua `spin[i] = +1`;
- Caso contrário, atribua `spin[i] = -1`.

Dessa forma, cada spin terá a mesma probabilidade de assumir os valores $+1$ ou -1 , gerando uma configuração inicial completamente desordenada.

3.2 Cálculo da Variação de Energia (ΔE)

Em vez de calcular a energia total do sistema a cada tentativa de inversão de spin (o que seria computacionalmente custoso), utilizamos uma abordagem local. Como apenas um spin é alterado por vez, basta calcular a variação de energia associada a esse sítio e seus vizinhos mais próximos.

Considerando um sítio i com spin s_i e seus quatro vizinhos mais próximos, temos:

$$E_a = -Js_i(s_{nn_0} + s_{nn_1} + s_{nn_2} + s_{nn_3})$$

Após inverter o spin ($s_i \rightarrow -s_i$), a nova energia local é:

$$E_n = -J(-s_i)(s_{nn_0} + s_{nn_1} + s_{nn_2} + s_{nn_3}) = +Js_i(s_{nn_0} + s_{nn_1} + s_{nn_2} + s_{nn_3})$$

Portanto, a variação de energia é dada por:

$$\Delta E = E_n - E_a = 2Js_i(s_{nn_0} + s_{nn_1} + s_{nn_2} + s_{nn_3})$$

Exercício 2 — Cálculo de ΔE

Implemente uma função `double calcula_deltaE(int i, double J)` que calcula a variação de energia associada à tentativa de inversão do spin no sítio i .

Para isso:

- Acesse os quatro vizinhos do sítio i utilizando a estrutura `nn[i][j]`, com $j = 0, 1, 2, 3$;
- Some os valores dos spins vizinhos: `spin[nn[i][j]]`;
- Utilize a expressão deduzida para ΔE ;
- Retorne o valor calculado.

3.3 Algoritmo de Metrópolis: Implementação

Com os módulos anteriores implementados, podemos agora integrar todas as etapas no algoritmo de Metrópolis. A cada passo, escolhemos um sítio aleatório da rede, calculamos a variação de energia associada à inversão do spin e decidimos se a mudança

será aceita ou não com base no critério de Metrópolis.

Exercício 3 — Implementação do Passo de Metrópolis

Implemente uma função `void passo_metropolis(double J, double T)` que realiza uma tentativa de atualização do sistema.

```
1 void passo_metropolis(double J, double T) {
2     int i = rand_sitio(); // Escolhe um sitio
3     aleatorio
4
5     // Calcula a variacao de energia associada a
6     inversao do spin
7     double delta_E = calcula_deltaE(i, J);
8
9     if (delta_E <= 0) {
10         // Aceita sempre: diminui (ou mantem) a
11         energia
12         spin[i] = -spin[i];
13     } else {
14         float alfa = rand_float();
15         double beta = 1.0 / T;
16
17         // Aceita com probabilidade de Boltzmann
18         if (alfa < exp(-beta * delta_E)) {
19             spin[i] = -spin[i];
20         }
21     }
22 }
```

3.4 Escala Temporal: Passo de Monte Carlo (PMC)

Na simulação, precisamos definir uma unidade de tempo adequada para acompanhar a evolução do sistema. Realizar apenas algumas tentativas de atualização (por exemplo, 5 chamadas do algoritmo de Metrópolis) não é suficiente para afetar significativamente todos os spins da rede.

Por isso, define-se o chamado **Passo de Monte Carlo (PMC)** como a unidade de tempo do algoritmo.

Um PMC corresponde a N tentativas de atualização, onde N é o número total de

spins do sistema:

$$1 \text{ PMC} = N \text{ tentativas de atualização.}$$

No caso de uma rede quadrada, temos $N = L \times L$.

Essa definição garante que, em média, cada spin tenha tido a oportunidade de ser atualizado uma vez por PMC.

Exercício 4 — Simulação Completa com PMC

Agora vamos integrar todos os componentes do algoritmo, incluindo:

- inicialização da rede;
- termalização (sem coleta de dados);
- regime estacionário (com coleta de observáveis).

```
1  int main() {
2      double J = 1.0;
3      double T = 2.0;
4      int pmc, k;
5      double mag_media = 0;
6
7      boundaries();          // Define as vizinhanças (
8                              // condicoes de contorno)
9      inicializa_rede();     // Ex. 1
10
11     // Etapa de termalizacao (atingir equilibrio)
12     for(pmc = 0; pmc < 10000; pmc++) {
13         for(k = 0; k < N; k++) {
14             passo_metropolis(J, T); // Ex. 3
15         }
16     }
17
18     // Regime estacionario (coleta de dados)
19     for(pmc = 0; pmc < 5000; pmc++) {
20         for(k = 0; k < N; k++) {
21             passo_metropolis(J, T);
22         }
23
24         // Exemplo: acumular magnetizacao
25         // mag_media += calcula_magnetizacao();
26     }
```

```
25         }  
26  
27         return 0;  
28     }
```

4 Resumo

O algoritmo de Metrópolis permite substituir o cálculo direto de grandezas globais por atualizações locais simples, incorporando corretamente os efeitos térmicos por meio de regras probabilísticas.

Com isso, conseguimos simular sistemas físicos complexos de forma eficiente, explorando suas propriedades estatísticas sem a necessidade de calcular explicitamente todas as configurações possíveis.